Improving Code Vulnerability Detection Using LLMs

Andrei Tudor Fărcășanu · Advisor: Traian-Eugen Rebedea
National University of Science and Technology POLITEHNICA Bucharest
tudor.farcasanu@stud.acs.upb.ro

Research question

Can a compact LLM learn to distinguish a vulnerable function from its subtly patched counterpart by learning explicit security reasoning and then optimizing against task-specific rewards?

28.38% Pair-wise Correct combined benchmark

25.93% Pair-wise Correct PrimeVul test

∼2× GPT-4's reported PrimeVul P-C

12,912 unique deduplicated training functions

Method at a glance

1. Consolidate data
PrimeVul + SVEN + SecVulEval
function-level deduplication

→

2. Generate reasoning
Qwen3-32B teacher
classification + important lines + rationale

→

3. Supervised fine-tuning
Full-parameter Qwen3-4B
hint-rich vs. hint-free ablations

→

4. Reinforcement learning
GRPO with classification, localization, format, and reward guards

Why paired evaluation matters

Standard accuracy can reward a model that predicts the same label for two nearly identical functions. PrimeVul instead evaluates a vulnerable function together with its patch:

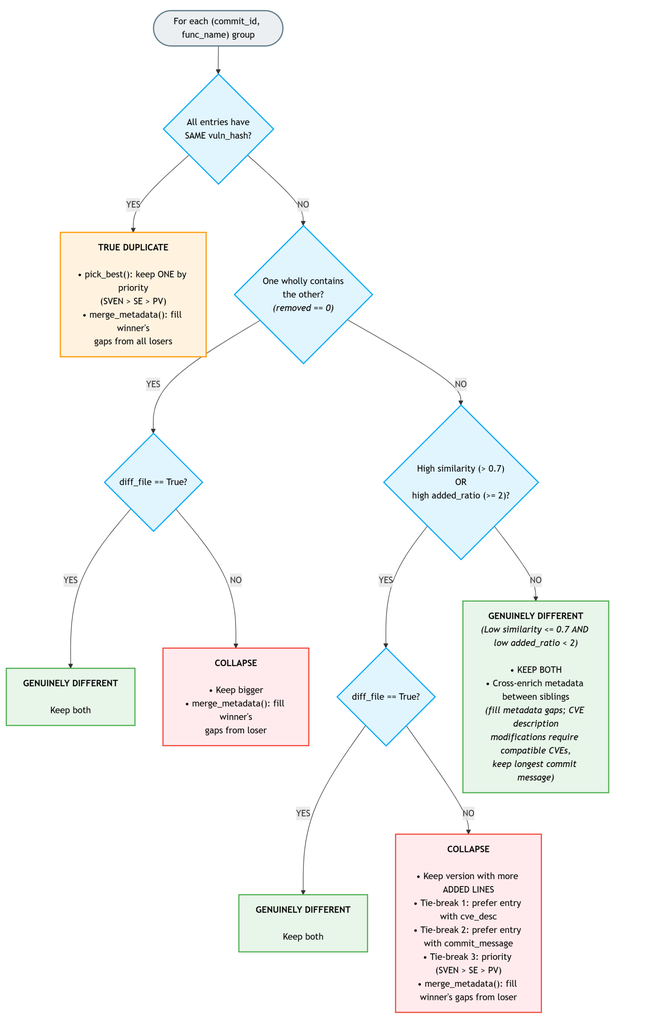
Vulnerable function ↔ **Patched** function

most times only different by a small semantic fix

P-C counts a pair only when *both* samples are classified correctly. P-V/P-B expose same-label collapse; P-R captures reversed labels.

Dataset consolidation

Commit-level deduplication removed valid cases whenever one commit patched multiple functions. Grouping by `(commit_id, func_name)` instead preserves distinct functions while collapsing true duplicates and enriching missing metadata.



Function-level deduplication: code hashes, normalized diffs, similarity, file paths, and source priority (SVEN > SecVulEval > PrimeVul).

Additional data quality work: invalid SecVulEval line indices were repaired using text search or diffs; non-fixes and uninformative CWE labels were removed; missing metadata was merged from compatible sibling records.

Consolidated training corpus

12,912 unique functions

6,208 unique CVEs

144 CWE categories

Prompt variations

- **Context selection:** commit messages were dropped when a CVE description was already available, or truncated to fit the token budget, preserving the useful security context while reducing redundant metadata
- **Reasoning dilution:** when a CVE description was already available, commit messages were dropped after the teacher was observed spending too much reasoning on metadata paraphrase

Reasoning-augmented distillation

The Qwen3-32B teacher sees line-numbered code and, when available, CVE/CWE context and changed-line hints. It produces a structured target:

```
{
  "classification": "VULNERABLE",
  "important_lines": [...],
  "reasoning_summary": "..."
}
```

The Qwen3-4B student therefore learns not only the label, but a code-level analysis linking untrusted inputs, missing checks, unsafe operations, and vulnerable lines.

GRPO reward designs

For each prompt, GRPO samples $G = 16$ candidate responses. We tested three discrete reward designs combining classification, line localization, and output validity:

Outcome	G1	G2	G3
Correct + ≥ 75% lines	1.00	**1.25**	1.00
Correct + ≥ 50% lines	1.00	1.00	1.00
Correct class only	0.60	0.75	0.75
Wrong + some correct lines	0.30	0.40	0.30
Valid JSON only	0.05	0.15	0.10
Invalid / incomplete	0.00	0.00	0.00
Patched-line scoring	yes	yes	no
Spray + outlier guards	no	no	**yes**

G1 produced the best P-C. G2's 1.25 tier raised training rewards but encouraged *line spraying*. G3 added a 60-line spray guard, capped ground-truth outliers, and stopped scoring patched-line localization; it improved behavioral stability, not peak discrimination.

What changed after fine-tuning?

Example: division by zero from file input (CWE-369)

```
bi->pixelsize = dbuf_getu16be(f, pos+18);
// no validation of pixelsize
bi->pdwidth = (bi->rowbytes * 8) / bi->pixelsize;
```

Base model: Assumes valid file-format data; predicts VULNERABLE.

SFT model: Treats file input as untrusted; identifies the zero divisor and predicts VULNERABLE.

The same qualitative improvement appears for unchecked iterators, missing parameter validation across calls, and out-of-bounds array indices. Resource-exhaustion cases remain difficult.

Experimental setup

- Student: Qwen3-4B, full-parameter SFT with DeepSpeed ZeRO-3 on 3×H100 GPUs; one epoch; learning rate 2×10^{-5}.
- GRPO: learning rate 10^{-6}, $G = 16$ responses per prompt, 60 prompts per iteration, and $\beta = 0$; rollouts used SGLang with top-p 0.95 and top-k 20.
- Training: full deduplicated corpus (24,440 training samples; 1,286 validation samples) on 3×H100 GPUs with FSDP2 and parameter/optimizer offload.
- Evaluation: 970 held-out vulnerability pairs (1,940 functions), including 407 PrimeVul test pairs.
- Reported main results use temperature $T = 0.6$, which generally produced the strongest P-C scores.

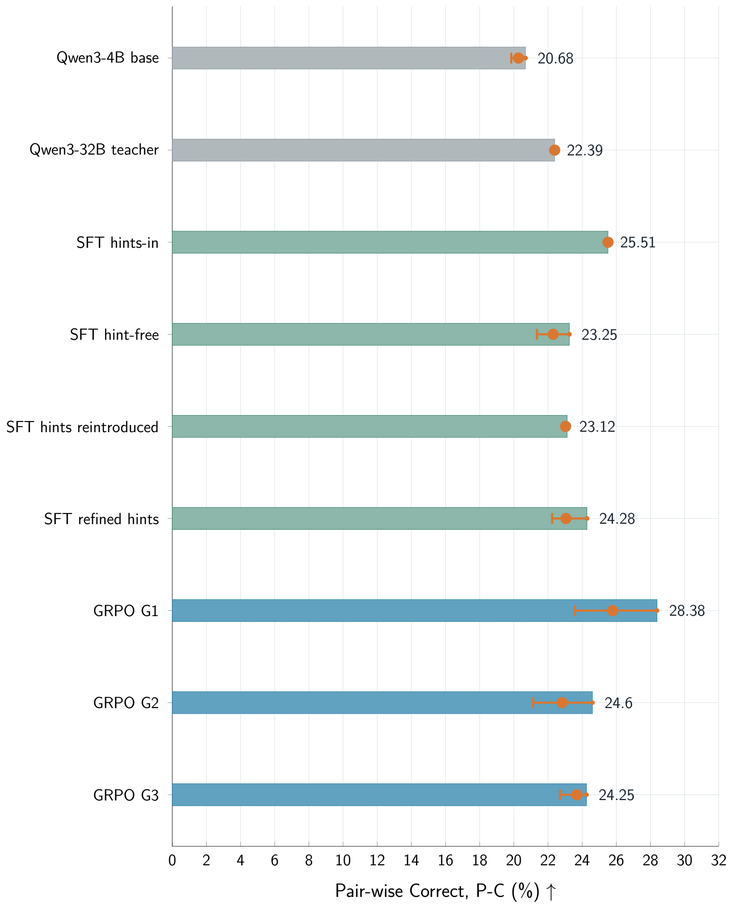
What changed after reinforcement learning?

Peak discrimination: GRPO Run 1 raised P-C from 23.25% for hint-free SFT to **28.38%** (+5.13 points), but retained a conservative bias: P-B remained 40.56% and only 1,693/1,940 outputs were parsable.

Stability trade-off: G3's spray and outlier guards improved parsability to 1,875/1,940 and produced a more balanced P-V/P-B profile, but its final-checkpoint P-C was 24.25%. Reward design therefore changed the behavior learned by RL, not just the score.

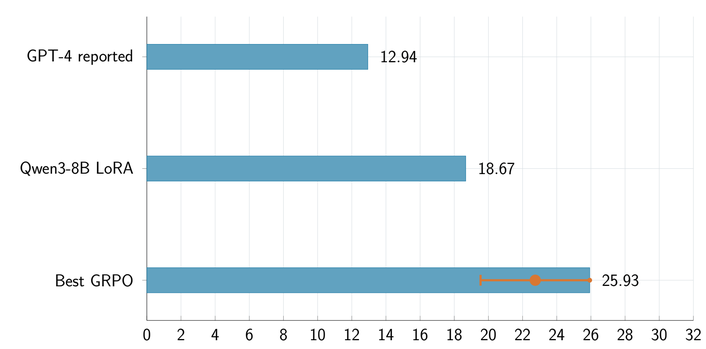
Combined benchmark: P-C across runs

Bar end = **best P-C**; orange dot = **average**; whisker = **best minus spread to best** across evaluation runs.



Combined held-out benchmark: 970 vulnerable/patched pairs. Single-run models have no visible spread. G1 is the original 4-tier reward; G2 is the 5-tier reward; G3 is the guarded reward.

PrimeVul test split



	Acc.↑	F1↑	P-C↑	P-V↓	P-B↓	P-R↓
Best GRPO	55.44	48.01	**25.93**	15.31	44.06	14.68

The whisker shows the observed range implied by Best − Spread through Best; the orange dot is the average. GPT-4 is the value reported by PrimeVul.

Ablations and experiment variations

- **Hint-free SFT** forces reasoning from code and provides the strongest GRPO starting point, but often fails to finish valid JSON (1,696/1,940 parsable before GRPO).
- **Hints-in-SFT** improves format compliance, but models become dependent on metadata and over-predict vulnerabilities when hints disappear at evaluation.
- **Reward exploitation** matters: rewarding high line overlap encouraged the model to predict too many lines; guards fixed the behavior but did not improve peak P-C.
- **Student surpasses teacher:** the best 4B student reaches 28.38% P-C versus 22.39% for the 32B teacher in zero-shot evaluation.

Conclusions and limitations

Reasoning-augmented SFT plus GRPO produces a measurable improvement beyond supervised learning alone and approximately doubles GPT-4's reported PrimeVul P-C result.

However, this is not deployment-ready. Peak performance has substantial run/temperature variance, parsability trades off against discrimination, and even the best model correctly resolves only about one quarter of vulnerability pairs.

Next steps: continuous rewards, explicit format rewards, static-analysis signals, code-specialized base models, and repository-level context.

Reasoning supervision provides the signal; GRPO strengthens paired discrimination; reward design determines whether the model learns the task or exploits the metric.